

# Übungsaufgabe Teil 3: Die ChickenFlix-Datenbank

Nachdem Sie nun Ihr theoretisches und praktisches Grundlagenwissen aufgefrischt haben, sind Sie endlich bereit, dieses auf die Unternehmensdaten Ihrer Streaming-Plattform „ChickenFlix“ anzuwenden.

Die bestehende Datenbankstruktur weist verschiedene Schwachstellen auf, die im Rahmen der folgenden Aufgaben analysiert und schrittweise überarbeitet werden sollen. Ziel ist die Entwicklung eines konsistenten und relational korrekt modellierten Datenbankschemas in 3NF.

---

---

## a) Datenvorbereitung

i. Lesen Sie die Daten aus *ChickenFlix\_Database.csv* als Pandas-Dataframe ein.

```
In [1]: import pandas as pd
```

```
In [2]: chicken_flix_df = pd.read_csv("ChickenFlix_Database.csv", sep=";", na_values=["NaN"])
chicken_flix_df
```

Out[2]:

|    | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                                  | Dauer | FilmBewertung | Watchtime |
|----|----------|-----------------------------------------------------|---------|-------|--------|--------------------------------------------|-------|---------------|-----------|
| 0  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 1      | Chicken Run - Hennen<br>rennen             | 84.0  | 7.1           | 83        |
| 1  | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 40        |
| 2  | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 3      | Chicken People                             | 83.0  | 7.0           | 83        |
| 3  | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 4      | Poultrygeist: Night of<br>the Chicken Dead | NaN   | NaN           | 1205      |
| 4  | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | NaN           | 90        |
| 5  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 88        |
| 6  | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 81        |
| 7  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 80        |
| 8  | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 3      | Chicken People                             | 83.0  | 7.0           | 50        |
| 9  | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 1      | Chicken Run - Hennen<br>rennen             | 84.0  | 7.1           | 82        |
| 10 | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 85        |
| 11 | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 4      | Poultrygeist: Night of<br>the Chicken Dead | NaN   | NaN           | 100       |
| 12 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 130       |
| 13 | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | 7.1           | 91        |

|    | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                         | Dauer | FilmBewertung | Watchtime |
|----|----------|-----------------------------------------------------|---------|-------|--------|-----------------------------------|-------|---------------|-----------|
| 14 | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 120       |
| 15 | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 3      | Chicken People                    | 83.0  | 7.0           | 80        |
| 16 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 80        |
| 17 | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 5      | Super Size Me 2: Holy<br>Chicken! | 93.0  | NaN           | 90        |
| 18 | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 3      | Chicken People                    | 83.0  | 7.0           | 82        |
| 19 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 2      | Himmel und Huhn                   | 81.0  | 5.7           | 70        |
| 20 | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 3      | Chicken People                    | 83.0  | 7.0           | 83        |

## ii. Prüfen Sie die vorliegenden Daten auf Vollständigkeit und Duplikate und bereinigen Sie den Datensatz entsprechend.

In der obigen Tabelle sehen wir bereits, dass NaN-Werte vorliegen.

Jetzt wollen wir systematischer suchen:

```
In [3]: chicken_flix_df[chicken_flix_df.isna().any(axis=1)]
```

| Out[3]: | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                                  | Dauer | FilmBewertung | Watchtime |
|---------|----------|-----------------------------------------------------|---------|-------|--------|--------------------------------------------|-------|---------------|-----------|
| 3       | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 4      | Poultrygeist: Night of<br>the Chicken Dead | NaN   | NaN           | 1205      |
| 4       | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | NaN           | 90        |
| 11      | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 4      | Poultrygeist: Night of<br>the Chicken Dead | NaN   | NaN           | 100       |
| 17      | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | NaN           | 90        |

Für den Film *Poultrygeist: Night of the Chicken Dead* ist weder Dauer noch FilmBewertung angegeben.  
Nachschlagen auf [www.imdb.com](http://www.imdb.com) --> Dauer: 103, FilmBewertung: 6.0

```
In [4]: chicken_flix_df.loc[3, "Dauer"]=103
chicken_flix_df.loc[3, "FilmBewertung"]=6.0

chicken_flix_df.loc[11, "Dauer"]=103
chicken_flix_df.loc[11, "FilmBewertung"]=6.0
```

Für den Film *Super Size Me 2: Holy Chicken!* fehlt an machen Stellen die FilmBewertung.  
Diese kann Zeile 13 entnommen werden --> FilmBewertung: 7.1.0

```
In [5]: chicken_flix_df.loc[13, "FilmBewertung"]
```

Out[5]: 7.1

Damit können die fehlenden Werte nun angepasst werden:

```
In [6]: chicken_flix_df.loc[4, "FilmBewertung"] = 7.1
chicken_flix_df.loc[17, "FilmBewertung"] = 7.1
```

Nochmals prüfen:

```
In [7]: chicken_flix_df[chicken_flix_df.isna().any(axis=1)]
```

```
Out[7]:
```

| NutzerID | Nutzer | AboTyp | Preis | FilmID | Filmtitel | Dauer | FilmBewertung | Watchtime |
|----------|--------|--------|-------|--------|-----------|-------|---------------|-----------|
|----------|--------|--------|-------|--------|-----------|-------|---------------|-----------|

--> Nun liegen keine NaN-Werte mehr vor. Andere Unvollständigkeiten sind in diesem Beispiel nicht gegeben.

**Jetzt prüfen wir noch auf Duplikate:**

```
In [8]: chicken_flix_df[chicken_flix_df.duplicated()]
```

```
Out[8]:
```

| NutzerID | Nutzer                                       | AboTyp | Preis | FilmID | Filmtitel      | Dauer | FilmBewertung | Watchtime |
|----------|----------------------------------------------|--------|-------|--------|----------------|-------|---------------|-----------|
| 20       | 3 Hildegard Huhn, Hildegard.Huhn@fakemail.de | Basic  | 7.99  | 3      | Chicken People | 83.0  | 7.0           | 83        |

Tatsächlich entspricht Zeile 20 genau Zeile 2:

```
In [9]: chicken_flix_df.iloc[[2]]
```

```
Out[9]:
```

| NutzerID | Nutzer                                       | AboTyp | Preis | FilmID | Filmtitel      | Dauer | FilmBewertung | Watchtime |
|----------|----------------------------------------------|--------|-------|--------|----------------|-------|---------------|-----------|
| 2        | 3 Hildegard Huhn, Hildegard.Huhn@fakemail.de | Basic  | 7.99  | 3      | Chicken People | 83.0  | 7.0           | 83        |

Zeile 20 kann also gelöscht werden:

```
In [10]: chicken_flix_df.drop_duplicates(inplace=True)  
chicken_flix_df
```

Out[10]:

|    | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                                  | Dauer | FilmBewertung | Watchtime |
|----|----------|-----------------------------------------------------|---------|-------|--------|--------------------------------------------|-------|---------------|-----------|
| 0  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 1      | Chicken Run - Hennen<br>rennen             | 84.0  | 7.1           | 83        |
| 1  | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 40        |
| 2  | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 3      | Chicken People                             | 83.0  | 7.0           | 83        |
| 3  | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 4      | Poultrygeist: Night of<br>the Chicken Dead | 103.0 | 6.0           | 1205      |
| 4  | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | 7.1           | 90        |
| 5  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 88        |
| 6  | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 81        |
| 7  | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 2      | Himmel und Huhn                            | 81.0  | 5.7           | 80        |
| 8  | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 3      | Chicken People                             | 83.0  | 7.0           | 50        |
| 9  | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 1      | Chicken Run - Hennen<br>rennen             | 84.0  | 7.1           | 82        |
| 10 | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 85        |
| 11 | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 4      | Poultrygeist: Night of<br>the Chicken Dead | 103.0 | 6.0           | 100       |
| 12 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 6      | Fried Chicken Day                          | 90.0  | 9.7           | 130       |
| 13 | 2        | Patricio Pollo,<br>Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 5      | Super Size Me 2: Holy<br>Chicken!          | 93.0  | 7.1           | 91        |

|    | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                         | Dauer | FilmBewertung | Watchtime |
|----|----------|-----------------------------------------------------|---------|-------|--------|-----------------------------------|-------|---------------|-----------|
| 14 | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 120       |
| 15 | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 3      | Chicken People                    | 83.0  | 7.0           | 80        |
| 16 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 80        |
| 17 | 1        | Pauline Poulet,<br>Pauline.Poulet@fakemail.de       | Premium | 12.99 | 5      | Super Size Me 2: Holy<br>Chicken! | 93.0  | 7.1           | 90        |
| 18 | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 3      | Chicken People                    | 83.0  | 7.0           | 82        |
| 19 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 2      | Himmel und Huhn                   | 81.0  | 5.7           | 70        |

iii. Ändern Sie den Namen von Nutzerin „Pauline Poulet“ in „Pauline Pute“ und passen sie die Mail-Adresse entsprechend an.

Manuelles Ändern der Einträge:

```
In [11]: chicken_flix_df.loc[0, "Nutzer"] = "Pauline Pute, Pauline.Pute@fakemail.de"
chicken_flix_df.loc[5, "Nutzer"] = "Pauline Pute, Pauline.Pute@fakemail.de"
chicken_flix_df.loc[7, "Nutzer"] = "Pauline Pute, Pauline.Pute@fakemail.de"
chicken_flix_df.loc[15, "Nutzer"] = "Pauline Pute, Pauline.Pute@fakemail.de"
chicken_flix_df.loc[17, "Nutzer"] = "Pauline Pute, Pauline.Pute@fakemail.de"

chicken_flix_df
```

Out[11]:

|    | NutzerID | Nutzer                                           | AboTyp  | Preis | FilmID | Filmtitel                               | Dauer | FilmBewertung | Watchtime |
|----|----------|--------------------------------------------------|---------|-------|--------|-----------------------------------------|-------|---------------|-----------|
| 0  | 1        | Pauline Pute, Pauline.Pute@fakemail.de           | Premium | 12.99 | 1      | Chicken Run - Hennen rennen             | 84.0  | 7.1           | 83        |
| 1  | 2        | Patricio Pollo, Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                         | 81.0  | 5.7           | 40        |
| 2  | 3        | Hildegard Huhn, Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 3      | Chicken People                          | 83.0  | 7.0           | 83        |
| 3  | 4        | Hahna Meier, Hahna.Meier@fakemail.de             | Premium | 12.99 | 4      | Poultrygeist: Night of the Chicken Dead | 103.0 | 6.0           | 1205      |
| 4  | 5        | Jacky Chickenwing, Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 5      | Super Size Me 2: Holy Chicken!          | 93.0  | 7.1           | 90        |
| 5  | 1        | Pauline Pute, Pauline.Pute@fakemail.de           | Premium | 12.99 | 6      | Fried Chicken Day                       | 90.0  | 9.7           | 88        |
| 6  | 3        | Hildegard Huhn, Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 2      | Himmel und Huhn                         | 81.0  | 5.7           | 81        |
| 7  | 1        | Pauline Pute, Pauline.Pute@fakemail.de           | Premium | 12.99 | 2      | Himmel und Huhn                         | 81.0  | 5.7           | 80        |
| 8  | 4        | Hahna Meier, Hahna.Meier@fakemail.de             | Premium | 12.99 | 3      | Chicken People                          | 83.0  | 7.0           | 50        |
| 9  | 5        | Jacky Chickenwing, Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 1      | Chicken Run - Hennen rennen             | 84.0  | 7.1           | 82        |
| 10 | 5        | Jacky Chickenwing, Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 6      | Fried Chicken Day                       | 90.0  | 9.7           | 85        |
| 11 | 2        | Patricio Pollo, Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 4      | Poultrygeist: Night of the Chicken Dead | 103.0 | 6.0           | 100       |
| 12 | 4        | Hahna Meier, Hahna.Meier@fakemail.de             | Premium | 12.99 | 6      | Fried Chicken Day                       | 90.0  | 9.7           | 130       |
| 13 | 2        | Patricio Pollo, Patricio.Pollo@fakemail.de       | Basic   | 7.99  | 5      | Super Size Me 2: Holy Chicken!          | 93.0  | 7.1           | 91        |



|    | NutzerID | Nutzer                                              | AboTyp  | Preis | FilmID | Filmtitel                         | Dauer | FilmBewertung | Watchtime |
|----|----------|-----------------------------------------------------|---------|-------|--------|-----------------------------------|-------|---------------|-----------|
| 14 | 3        | Hildegard Huhn,<br>Hildegard.Huhn@fakemail.de       | Basic   | 7.99  | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 120       |
| 15 | 1        | Pauline Pute, Pauline.Pute@fakemail.de              | Premium | 12.99 | 3      | Chicken People                    | 83.0  | 7.0           | 80        |
| 16 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 1      | Chicken Run - Hennen<br>rennen    | 84.0  | 7.1           | 80        |
| 17 | 1        | Pauline Pute, Pauline.Pute@fakemail.de              | Premium | 12.99 | 5      | Super Size Me 2: Holy<br>Chicken! | 93.0  | 7.1           | 90        |
| 18 | 5        | Jacky Chickenwing,<br>Jacky.Chickenwing@fakemail.de | Basic   | 7.99  | 3      | Chicken People                    | 83.0  | 7.0           | 82        |
| 19 | 4        | Hahna Meier,<br>Hahna.Meier@fakemail.de             | Premium | 12.99 | 2      | Himmel und Huhn                   | 81.0  | 5.7           | 70        |

**Wichtig: Wird eine Ersetzung vergessen, liegt eine Einfüge-Anomalie vor und die Daten sind nicht mehr konsistent.**

Zwar existieren effizientere und elegantere Methoden zur Suche und gezielten Anpassung von Datensätzen in Pandas. An dieser Stelle soll jedoch bewusst auf eine manuelle Vorgehensweise zurückgegriffen werden, um die Problematik redundanter Daten und die daraus resultierenden Fehleranfälligkeiten besser nachvollziehen zu können.

## b) Überführung in erste Normalform (1NF)

### i. Überführen Sie die Daten in erste Normalform.

Für das Attribut *Nutzer* liegen die Werte nicht atomar vor.

Besser Aufteilung in *Vorname*, *Nachname* und *Mail*:

Aufsplitten von Attribut *Nutzer* bei "," --> *Name* und *Mail*

```
In [12]: chicken_flix_df[["Name", "Mail"]] = chicken_flix_df["Nutzer"].str.split(" ", expand=True)
```

Die Spalte *Mail* wird zunächst an das Ende des Dataframes angehängt:

```
In [13]: chicken_flix_df.head(1)
```

```
Out[13]:
```

|   | NutzerID | Nutzer                                    | AboTyp  | Preis | FilmID | Filmtitel                            | Dauer | FilmBewertung | Watchtime | Name            | Mail                     |
|---|----------|-------------------------------------------|---------|-------|--------|--------------------------------------|-------|---------------|-----------|-----------------|--------------------------|
| 0 | 1        | Pauline Pute,<br>Pauline.Pute@fakemail.de | Premium | 12.99 | 1      | Chicken<br>Run -<br>Hennen<br>rennen | 84.0  | 7.1           | 83        | Pauline<br>Pute | Pauline.Pute@fakemail.de |



Aufsplitten von Attribut *Name* bei " " --> *Vorname* und *Nachname*:

```
In [14]: chicken_flix_df[["Vorname", "Nachname"]] = chicken_flix_df["Name"].str.split(" ", n=1, expand=True)
```

Die Spalten *Vorname* und *Nachname* werden ebenfalls an das Ende des Dataframes angehängt:

```
In [15]: chicken_flix_df.head(1)
```

```
Out[15]:
```

|   | NutzerID | Nutzer                                    | AboTyp  | Preis | FilmID | Filmtitel                            | Dauer | FilmBewertung | Watchtime | Name            | Mail                     | Vorname | Nachname |
|---|----------|-------------------------------------------|---------|-------|--------|--------------------------------------|-------|---------------|-----------|-----------------|--------------------------|---------|----------|
| 0 | 1        | Pauline Pute,<br>Pauline.Pute@fakemail.de | Premium | 12.99 | 1      | Chicken<br>Run -<br>Hennen<br>rennen | 84.0  | 7.1           | 83        | Pauline<br>Pute | Pauline.Pute@fakemail.de | Pauline | Pute     |



Gewünschte Spalten auswählen (die Spalte *Nutzer* wird nicht mehr benötigt):

```
In [16]: chicken_flix_df = chicken_flix_df[["NutzerID", "Vorname", "Nachname", "Mail", "AboTyp", "Preis", "FilmID", "Filmtitel", "Dauer"]]
```

```
In [17]: chicken_flix_df.head(1)
```

```
Out[17]:
```

|   | NutzerID | Vorname | Nachname | Mail                     | AboTyp  | Preis | FilmID | Filmtitel                         | Dauer | FilmBewertung | Watchtime |
|---|----------|---------|----------|--------------------------|---------|-------|--------|-----------------------------------|-------|---------------|-----------|
| 0 | 1        | Pauline | Pute     | Pauline.Pute@fakemail.de | Premium | 12.99 | 1      | Chicken Run -<br>Hennen<br>rennen | 84.0  | 7.1           | 83        |

Jetzt liegen die Daten atomar und somit in erster Normalform (1NF) vor.

## c) Überführung in zweite Normalform (2NF)

i. Untersuchen Sie die Attribute *Mail* und *AboTyp* auf Redundanzen.

```
In [18]: chicken_flix_df["Mail"].value_counts()
```

```
Out[18]: Mail
Pauline.Pute@fakemail.de      5
Hahna.Meier@fakemail.de      5
Jacky.Chickenwing@fakemail.de 4
Patricio.Pollo@fakemail.de   3
Hildegard.Huhn@fakemail.de   3
Name: count, dtype: int64
```

```
In [19]: chicken_flix_df["AboTyp"].value_counts()
```

```
Out[19]: AboTyp
Premium    10
Basic      10
Name: count, dtype: int64
```

--> In beiden Attributen liegen Redundanzen vor. Es gibt noch weitere Redundanzen im Datensatz, z.B. in *Vorname* und *Nachname*

ii. Bestimmen Sie einen geeigneten Schlüssel für die vorliegenden Daten.

Da jede Zeile eine Interaktion zwischen Nutzer und Film beschreibt bietet sich **(NutzerID, FilmID)** an.

### iii. Überführen Sie die Tabelle in zweite Normalform.

Es gilt:

- **Benutzerdaten:** *NutzerID* → *Vorname, Nachname, Mail, AboTyp*
- **Filmdaten:** *FilmID* → *Filmtitel, Dauer, FilmBewertung*
- **Nutzungsdaten:** (*NutzerID, FilmID*) → *Watchtime*

Hieraus ergeben sich die drei Relationen:

- Nur von *NutzerID* abhängig: *Vorname, Nachname, Mail, AboTyp, Preis*
- Nur von *FilmID* abhängig: *Filmtitel, Dauer, FilmBewertung*
- Vom Gesamtschlüssel (*NutzerID, FilmID*) abhängig: *Watchtime*

#### Anlegen der entsprechenden Tabellen:

```
In [20]: user_df = chicken_flix_df[["NutzerID", "Vorname", "Nachname", "Mail", "AboTyp", "Preis"]].drop_duplicates()
user_df
```

```
Out[20]:
```

|   | NutzerID | Vorname   | Nachname    | Mail                          | AboTyp  | Preis |
|---|----------|-----------|-------------|-------------------------------|---------|-------|
| 0 | 1        | Pauline   | Pute        | Pauline.Pute@fakemail.de      | Premium | 12.99 |
| 1 | 2        | Patricio  | Pollo       | Patricio.Pollo@fakemail.de    | Basic   | 7.99  |
| 2 | 3        | Hildegard | Huhn        | Hildegard.Huhn@fakemail.de    | Basic   | 7.99  |
| 3 | 4        | Hahna     | Meier       | Hahna.Meier@fakemail.de       | Premium | 12.99 |
| 4 | 5        | Jacky     | Chickenwing | Jacky.Chickenwing@fakemail.de | Basic   | 7.99  |

```
In [21]: film_df = chicken_flix_df[["FilmID", "Filmtitel", "Dauer", "FilmBewertung"]].drop_duplicates()
film_df
```

Out[21]:

|   | FilmID | Filmtitel                               | Dauer | FilmBewertung |
|---|--------|-----------------------------------------|-------|---------------|
| 0 | 1      | Chicken Run - Hennen rennen             | 84.0  | 7.1           |
| 1 | 2      | Himmel und Huhn                         | 81.0  | 5.7           |
| 2 | 3      | Chicken People                          | 83.0  | 7.0           |
| 3 | 4      | Poultrygeist: Night of the Chicken Dead | 103.0 | 6.0           |
| 4 | 5      | Super Size Me 2: Holy Chicken!          | 93.0  | 7.1           |
| 5 | 6      | Fried Chicken Day                       | 90.0  | 9.7           |

```
In [22]: nutzung_df = chicken_flix_df[["NutzerID", "FilmID", "Watchtime"]].drop_duplicates()  
nutzung_df
```

Out[22]:

|    | NutzerID | FilmID | Watchtime |
|----|----------|--------|-----------|
| 0  | 1        | 1      | 83        |
| 1  | 2        | 2      | 40        |
| 2  | 3        | 3      | 83        |
| 3  | 4        | 4      | 1205      |
| 4  | 5        | 5      | 90        |
| 5  | 1        | 6      | 88        |
| 6  | 3        | 2      | 81        |
| 7  | 1        | 2      | 80        |
| 8  | 4        | 3      | 50        |
| 9  | 5        | 1      | 82        |
| 10 | 5        | 6      | 85        |
| 11 | 2        | 4      | 100       |
| 12 | 4        | 6      | 130       |
| 13 | 2        | 5      | 91        |
| 14 | 3        | 1      | 120       |
| 15 | 1        | 3      | 80        |
| 16 | 4        | 1      | 80        |
| 17 | 1        | 5      | 90        |
| 18 | 5        | 3      | 82        |
| 19 | 4        | 2      | 70        |

Jetzt hängt kein Nichtschlüsselattribut mehr von einem Teil des Primärschlüssels ab und somit liegen die Daten in zweiter Normalform (2NF) vor.

---

## d) Überführung in dritte Normalform (3NF)

### i. Analysieren Sie die Daten auf transitive Abhängigkeiten.

Es gilt:

- Vertragsdaten: *AboTyp* → *Preis*

### ii. Überführen Sie die Tabelle in dritte Normalform.

**Anlegen der entsprechenden Tabelle:**

```
In [23]: abo_df = user_df[["AboTyp", "Preis"]].drop_duplicates()  
abo_df
```

```
Out[23]:
```

|          | <b>AboTyp</b> | <b>Preis</b> |
|----------|---------------|--------------|
| <b>0</b> | Premium       | 12.99        |
| <b>1</b> | Basic         | 7.99         |

**Entfernen des Preises aus User-Dataframe:**

```
In [24]: user_df = user_df[["NutzerID", "Vorname", "Nachname", "Mail", "AboTyp"]].drop_duplicates()  
user_df
```

Out[24]:

|   | NutzerID | Vorname   | Nachname    | Mail                          | AboTyp  |
|---|----------|-----------|-------------|-------------------------------|---------|
| 0 | 1        | Pauline   | Pute        | Pauline.Pute@fakemail.de      | Premium |
| 1 | 2        | Patricio  | Pollo       | Patricio.Pollo@fakemail.de    | Basic   |
| 2 | 3        | Hildegard | Huhn        | Hildegard.Huhn@fakemail.de    | Basic   |
| 3 | 4        | Hahna     | Meier       | Hahna.Meier@fakemail.de       | Premium |
| 4 | 5        | Jacky     | Chickenwing | Jacky.Chickenwing@fakemail.de | Basic   |

Jetzt liegen keine transitiven Abhängigkeiten mehr vor. Somit liegen die Daten in dritter Normalform (3NF) vor.

## e) Anlegen der Datenbank in SQLite

i. Legen Sie das Datenbankschema für die Relationen der 3NF an und übertragen Sie die Daten.

```
In [25]: import sqlite3

conn = sqlite3.connect("ChickenFlix.db")
cursor = conn.cursor()
```

**Relation *Nutzer*:**

```
In [26]: cursor.execute("""
CREATE TABLE IF NOT EXISTS NUTZER (
    NutzerID INTEGER PRIMARY KEY,
    Vorname VARCHAR(50),
    Nachname VARCHAR(50),
    Mail VARCHAR(50),
    AboTyp VARCHAR(50)
)
""")
```



Out[26]: <sqlite3.Cursor at 0x1f03b52a740>

```
In [27]: user_df.to_sql("NUTZER", conn, if_exists="replace", index=False)
```

Out[27]: 5

#### **Relation *Abo*:**

```
In [28]: cursor.execute("""
CREATE TABLE IF NOT EXISTS ABO (
    AboTyp VARCHAR(50) PRIMARY KEY,
    Preis REAL
)
""")
```

Out[28]: <sqlite3.Cursor at 0x1f03b52a740>

```
In [29]: abo_df.to_sql("ABO", conn, if_exists="replace", index=False)
```

Out[29]: 2

#### **Relation *Film*:**

```
In [30]: cursor.execute("""
CREATE TABLE IF NOT EXISTS FILM (
    FilmID INTEGER PRIMARY KEY,
    Filmtitel VARCHAR(50),
    Dauer REAL,
    FilmBewertung REAL
)
""")
```

Out[30]: <sqlite3.Cursor at 0x1f03b52a740>

```
In [31]: film_df.to_sql("FILM", conn, if_exists="replace", index=False)
```

Out[31]: 6

### Relation *Nutzung*:

```
In [32]: cursor.execute("""
CREATE TABLE IF NOT EXISTS NUTZUNG (
    NutzerID INTEGER,
    FilmID INTEGER,
    Watchtime REAL,

    PRIMARY KEY (NutzerID, FilmID),

    FOREIGN KEY (NutzerID)
        REFERENCES NUTZER(NutzerID),

    FOREIGN KEY (FilmID)
        REFERENCES FILM(FilmID)
)
""")
```

```
Out[32]: <sqlite3.Cursor at 0x1f03b52a740>
```

```
In [33]: nutzung_df.to_sql("NUTZUNG", conn, if_exists="replace", index=False)
```

```
Out[33]: 20
```

ii. Ändern Sie den Namen von Nutzerin „Pauline Pute“ zurück in „Pauline Poulet“ und passen sie die Mail-Adresse entsprechend an.

Anzeigen der Relation *Nutzer*:

```
In [34]: query =("""
SELECT *
FROM NUTZER;
""")

result = cursor.execute(query).fetchall()
result
```

```
Out[34]: [(1, 'Pauline', 'Pute', 'Pauline.Pute@fakemail.de', 'Premium'),
          (2, 'Patricio', 'Pollo', 'Patricio.Pollo@fakemail.de', 'Basic'),
          (3, 'Hildegard', 'Huhn', 'Hildegard.Huhn@fakemail.de', 'Basic'),
          (4, 'Hahna', 'Meier', 'Hahna.Meier@fakemail.de', 'Premium'),
          (5, 'Jacky', 'Chickenwing', 'Jacky.Chickenwing@fakemail.de', 'Basic')]
```

Ändern des *Nachnamen* und der *Mail* für Pauline Pute/Poulet:

(Hier ist nur ein einziger UPDATE-Befehl notwendig)

```
In [35]: cursor.execute("""
UPDATE NUTZER
SET Nachname = 'Poulet',
    Mail = 'pauline.poulet@example.com'
WHERE Vorname = 'Pauline' AND Nachname = 'Pute';
""")
```

```
Out[35]: <sqlite3.Cursor at 0x1f03b52a740>
```

Erneutes Anzeigen der Relation *Nutzer*:

```
In [36]: query =("""
SELECT *
FROM NUTZER;
""")

result = cursor.execute(query).fetchall()
result
```

```
Out[36]: [(1, 'Pauline', 'Poulet', 'pauline.poulet@example.com', 'Premium'),
          (2, 'Patricio', 'Pollo', 'Patricio.Pollo@fakemail.de', 'Basic'),
          (3, 'Hildegard', 'Huhn', 'Hildegard.Huhn@fakemail.de', 'Basic'),
          (4, 'Hahna', 'Meier', 'Hahna.Meier@fakemail.de', 'Premium'),
          (5, 'Jacky', 'Chickenwing', 'Jacky.Chickenwing@fakemail.de', 'Basic')]
```

---

---

## e) Datenanalyse mit SQL

i. Lassen Sie sich alle Filme mit *FilmID*, *FilmTitel* und *Dauer* ausgeben.

```
In [37]: query =("""
SELECT FilmID, Filmtitel, Dauer
FROM FILM;
""")

result = cursor.execute(query).fetchall()
result
```

```
Out[37]: [(1, 'Chicken Run - Hennen rennen', 84.0),
(2, 'Himmel und Huhn', 81.0),
(3, 'Chicken People', 83.0),
(4, 'Poultrygeist: Night of the Chicken Dead', 103.0),
(5, 'Super Size Me 2: Holy Chicken!', 93.0),
(6, 'Fried Chicken Day', 90.0)]
```

ii. Lassen Sie sich alle Nutzer mit dem *AboTyp* "Premium" ausgeben.

```
In [38]: query =("""
SELECT Vorname, Nachname
FROM NUTZER
WHERE AboTyp = 'Premium';
""")

result = cursor.execute(query).fetchall()
result
```

```
Out[38]: [('Pauline', 'Poulet'), ('Hahna', 'Meier')]
```

iii. Berechnen Sie die durchschnittliche Filmdauer.

```
In [39]: query =("""
SELECT AVG(Dauer) AS Durchschnittsdauer
FROM FILM;
""")
```

```
result = cursor.execute(query).fetchall()
result
```

Out[39]: [(89.0,)]

#### iv. Analysieren Sie wie oft jeder Film angesehen wurde.

```
In [40]: query =("""
SELECT F.Filmtitel, COUNT(*) AS AnzahlViews
FROM NUTZUNG N
JOIN FILM F ON N.FilmID = F.FilmID
GROUP BY F.Filmtitel;
""")

result = cursor.execute(query).fetchall()
result
```

Out[40]: [('Chicken People', 4),  
('Chicken Run - Hennen rennen', 4),  
('Fried Chicken Day', 3),  
('Himmel und Huhn', 4),  
('Poultrygeist: Night of the Chicken Dead', 2),  
('Super Size Me 2: Holy Chicken!', 3)]

#### v. Analysieren welche Filme Pauline Poulet angesehen hat?

```
In [41]: query =("""
SELECT U.Vorname, U.Nachname, F.Filmtitel
FROM NUTZUNG N
JOIN NUTZER U ON N.NutzerID = U.NutzerID
JOIN FILM F ON N.FilmID = F.FilmID
WHERE U.Vorname = 'Pauline'
      AND U.Nachname = 'Poulet';
""")

result = cursor.execute(query).fetchall()
result
```

```
Out[41]: [('Pauline', 'Poulet', 'Chicken Run - Hennen rennen'),  
          ('Pauline', 'Poulet', 'Himmel und Huhn'),  
          ('Pauline', 'Poulet', 'Chicken People'),  
          ('Pauline', 'Poulet', 'Super Size Me 2: Holy Chicken!'),  
          ('Pauline', 'Poulet', 'Fried Chicken Day')]
```